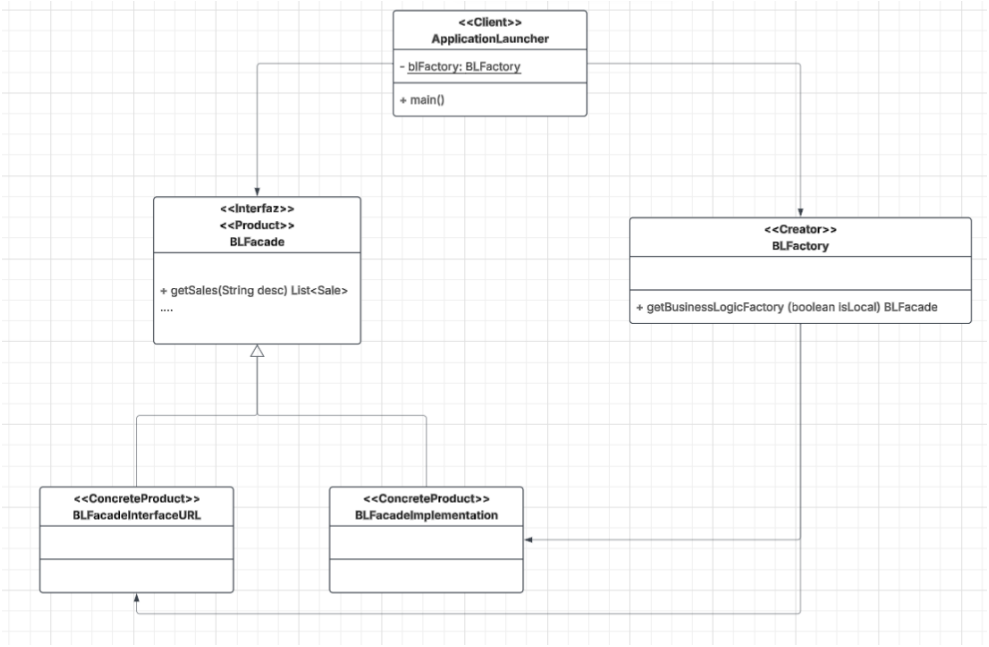
Miembros: Iker Perez, Aday Pérez

url github: https://github.com/iperez288/market26

1. Patrón Factory Method
   Diagrama UML



   En este problema, se crea una clase Factory la cual es la encargada de aplicar la lógica de creación de objetos, para ello tiene sus condiciones dentro y solo se encarga de eso. Esta devuelve un objeto de la interfaz BLFacade, y será usado por el application launcher

Código nuevo:

Clase ApplicationLauncher

```
public class ApplicationLauncher {

    private static final BLFactory blFactory=new BLFactory();

    public static void main(String[] args) {

        ConfigXML c=ConfigXML.getInstance();
        Locale.setDefault(new Locale(c.getLocale()));
        MainGUI a=new MainGUI("seller3@gmail.com");
        a.setVisible(true);

        try {

            UIManager.setLookAndFeel("javax.swing.plaf.metal.MetalLookAndFeel");

            BLFacade appFacadeInterface=blFactory.getBusinessLogicFactory(c.isBusinessLogicLocal());

            MainGUI.setBussinessLogic(appFacadeInterface);

        }catch (Exception e) {
            a.jLabelSelectOption.setText("Error: "+e.toString());
            a.jLabelSelectOption.setForeground(Color.RED);

            System.out.println("Error in ApplicationLauncher: "+e.toString());
        }
    }
}
```

Se ha eliminado el proceso de creación de BLFacade del método main para delegarlo a otra clase. De esta manera, únicamente debe llamar al método

getBusinessLogicFactory de la factoría BLFactory para que asi nos devuelva el objeto que corresponde.

Clase BLFactory:

```java
package businessLogic;

import java.net.URL;

public class BLFactory {

    public BLFacade getBusinessLogicFactory(boolean isLocal) {
        try {
            if (isLocal) {
                DataAccess da = new DataAccess();
                return new BLFacadeImplementation(da);
            } else {
                ConfigXML config = ConfigXML.getInstance();
                String serviceName = "http://" + config.getBusinessLogicNode() + ":"
                        + config.getBusinessLogicPort() + "/ws/"
                        + config.getBusinessLogicName() + "?wsdl";
                URL url = new URL(serviceName);

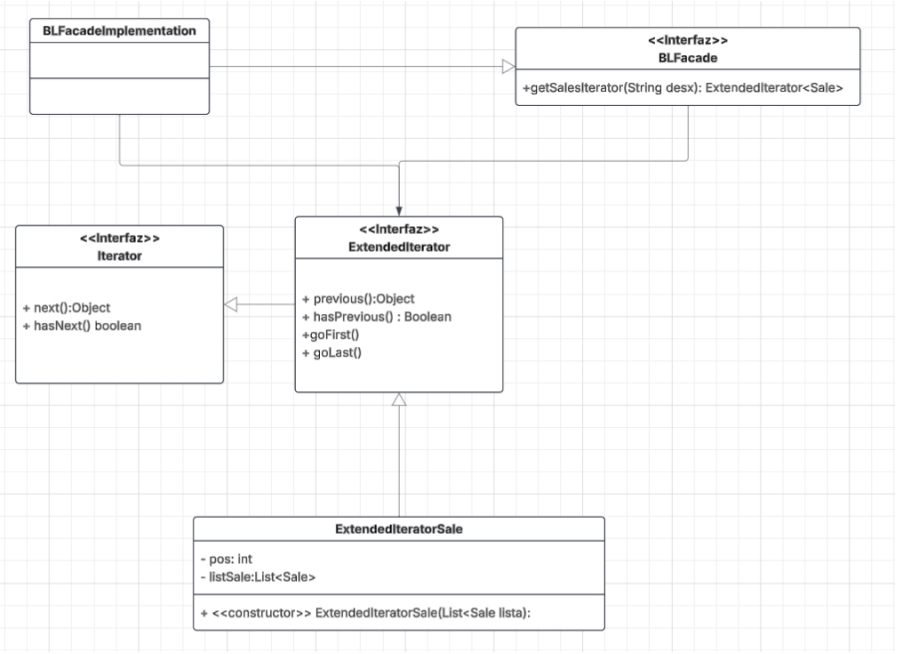
                QName qname = new QName("http://businessLogic/", "BLFacadeImplementationService");
                Service service = Service.create(url, qname);

                return service.getPort(BLFacade.class);
            }
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }
}
```

Se crea una clase BLFactory, encargada del proceso de crear los objetos, siguiendo la misma estructura de creación que existia originalmente solo que ahora se devolvera el objeto obtenido. Este recibe un parametro booleano para indicar si es local o en remoto y según eso actuara de una forma u otra.

2. Patrón Iterator

   Diagrama UML:



Para poder recorrer la lista se ha añadido a la interfaz BLFacade el metodo getSalesIterator(String desc), el cual es implementado por la BLFacadeImplementation reutilizando getSales(desc). Ambas clases usan la interfaz nueva ExtendedIterator, la cual extiende a Iterator, y es implementada por ExtendedIteratorSale.

```java
@WebService
public interface BLFacade  {


    public  ExtendedIterator<Sale>  getSalesIterator(String desc);

    /**
```

```java
public ExtendedIterator<Sale> getSalesIterator(String desc){
    return new ExtendedIteratorSale(getSales(desc));
}

/**
```

```java
public class ExtendedIteratorSale implements ExtendedIterator {

    private int pos;
    private List<Sale> listSale;

    public ExtendedIteratorSale (List<Sale> lista) {
        listSale=lista;
        this.pos=0;
    }
    @Override
    public boolean hasNext() {
        if(pos>=listSale.size()) return false;
        return true;
    }

    @Override
    public Object next() {
        Sale obj=listSale.get(pos);
        pos++;
        return obj;
    }

    @Override
    public Object previous() {
        pos--;
        Sale obj=listSale.get(pos);
        return obj;
    }

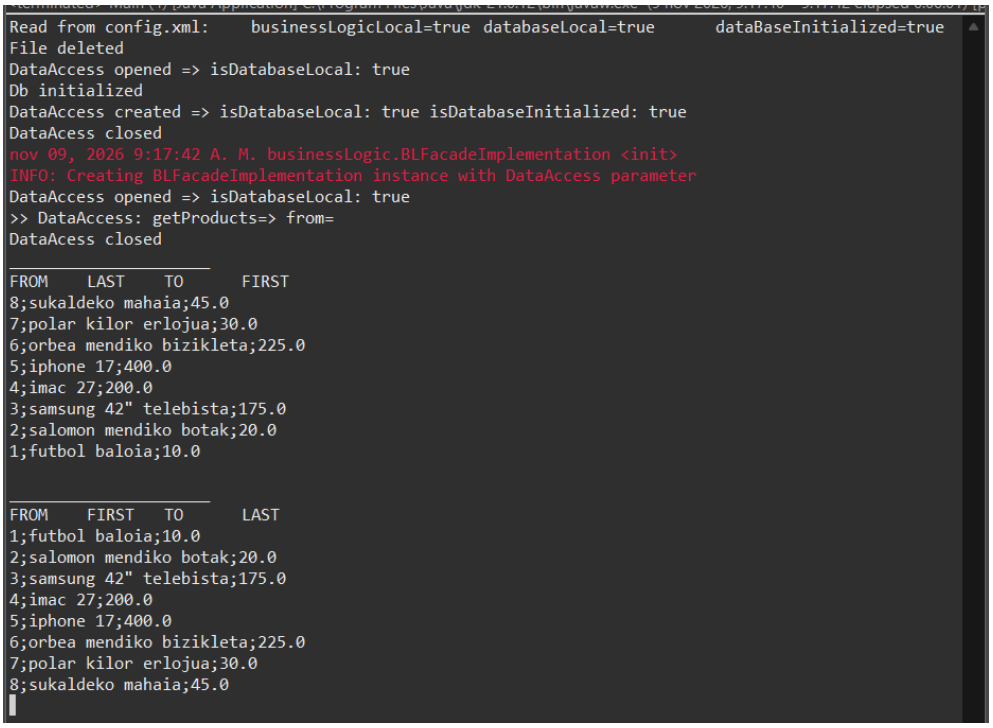
    @Override
    public boolean hasPrevious() {
        if(pos<=0) return false;
        return true;
    }

    @Override
    public void goFirst() {
        // TODO Auto-generated method stub
        pos=0;
    }

    @Override
    public void goLast() {
        // TODO Auto-generated method stub
        pos=listSale.size();
    }

}
```

Se ha creado una nueva clase, ExtendedIteratorSale, que implementa la interfaz ExtendedIterator. Esta clase se encarga de la lógica de recorrer la lista de objetos Sale con un puntero de posición (pos) y la lista de objetos, implementado los métodos hasPrevious(), previous(), goFirst() y goLast(), ademas de los de la interfaz Iterator, hasnext() y next().



```
Read from config.xml:    businessLogicLocal=true databaseLocal=true      dataBaseInitialized=true
File deleted
DataAccess opened => isDatabaseLocal: true
Db initialized
DataAccess created => isDatabaseLocal: true isDatabaseInitialized: true
DataAccess closed
nov 09, 2026 9:17:42 A. M. businessLogic.BLFacadeImplementation <init>
INFO: Creating BLFacadeImplementation instance with DataAccess parameter
DataAccess opened => isDatabaseLocal: true
>> DataAccess: getProducts=> from=
DataAcess closed
_____________________
FROM    LAST    TO      FIRST
8;sukaldeko mahaia;45.0
7;polar kilor erlojua;30.0
6;orbea mendiko bizikleta;225.0
5;iphone 17;400.0
4;imac 27;200.0
3;samsung 42" telebista;175.0
2;salomon mendiko botak;20.0
1;futbol baloia;10.0

_____________________
FROM    FIRST   TO      LAST
1;futbol baloia;10.0
2;salomon mendiko botak;20.0
3;samsung 42" telebista;175.0
4;imac 27;200.0
5;iphone 17;400.0
6;orbea mendiko bizikleta;225.0
7;polar kilor erlojua;30.0
8;sukaldeko mahaia;45.0
```

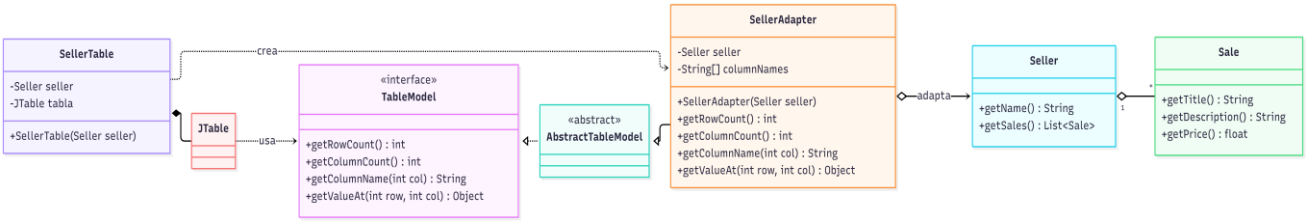


```java
public class MainIterator {
    public static void main(String[]    args)   {
//      the BL  is  local
    boolean isLocal =   true;
    BLFacade    blFacade =  new BLFactory().getBusinessLogicFactory(isLocal);
    ExtendedIterator<Sale>  it =    blFacade.getSalesIterator("");
    Sale s;
    System.out.println("_____________________");
    System.out.println("FROM    LAST    TO  FIRST");
    it.goLast();    //  Go  to  last    element
    while (it.hasPrevious())    {
    s = it.previous();
    System.out.println(s);
    }
    System.out.println();
    System.out.println("_____________________");
    System.out.println("FROM    FIRST   TO  LAST");
    it.goFirst();   //  Go  to  first   element
    while (it.hasNext())    {
    s = it.next();
    System.out.println(s);
    }
    }
}
```

El método main y su salida por pantalla nos muestra que la lista se recorre tanto de delante para atrás como en orden inverso.

3. Patrón Adapter

Diagrama UML:



Problema y solución

Se quiere mostrar en una JTable todas las ofertas (Sale) de un vendedor (Seller). Una JTable no sabe trabajar directamente con un objeto Seller, sino que obtiene sus datos a través de la interfaz TableModel (filas, columnas, nombres de columna y valor de cada celda). Como Seller y TableModel tienen interfaces incompatibles, y no queremos modificar la clase Seller del dominio, se aplica el patrón Adapter: se crea la clase SellerAdapter, que traduce las llamadas de la JTable (getRowCount, getValueAt...) a llamadas sobre el Seller (getSales().size(), getSales().get(i).getTitle()...).

Se ha usado la variante de adaptador de objetos (por composición): SellerAdapter contiene una referencia al Seller y hereda de AbstractTableModel, que ya implementa la parte común de TableModel (gestión de listeners, etc.). Así solo hay que implementar los métodos específicos.

En este caso la clase SellerTable es el cliente o el que usa los objetos, Table Model el objetivo a usar por el cliente, SellerAdapter el adaptador de la clase a usar, en este caso Seller.

SellerAdapter.java:

```java
public class SellerAdapter extends AbstractTableModel {

    private static final long serialVersionUID = 1L;

    private final Seller seller;
    private final String[] columnNames = { "title", "description", "price" };

    public SellerAdapter(Seller seller) {
        this.seller = seller;
    }

    @Override
    public int getRowCount() {
        return seller.getSales().size();
    }

    @Override
    public int getColumnCount() {
        return columnNames.length;
    }

    @Override
    public String getColumnName(int column) {
        return columnNames[column];
    }

    @Override
    public Class<?> getColumnClass(int column) {
        return column == 2 ? Float.class : String.class;
    }

    @Override
    public Object getValueAt(int row, int column) {
        Sale s = seller.getSales().get(row);
        switch (column) {
            case 0: return s.getTitle();
            case 1: return s.getDescription();
            case 2: return s.getPrice();
            default: return null;
        }
    }
}
```

SellerTable.java:

```java
public class SellerTable extends JFrame {

    private static final long serialVersionUID = 1L;
    private Seller seller;
    private JTable tabla;

    public SellerTable(Seller seller) {
        super(seller.getName() + "'s sales ");
        this.setBounds(100, 100, 700, 200);
        this.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        this.seller = seller;

        SellerAdapter adapt = new SellerAdapter(seller);
        tabla = new JTable(adapt);
        tabla.setPreferredScrollableViewportSize(new Dimension(500, 70));

        JScrollPane scrollPane = new JScrollPane(tabla);
        getContentPane().add(scrollPane, BorderLayout.CENTER);
    }
}
```

SellerTableMain.java:

```java
package gui;

import businessLogic.BLFacade;
import businessLogic.BLFactory;
import domain.Seller;

public class SellerTableMain {

    public static void main(String[] args) {
        // la lógica de negocio es local
        boolean isLocal = true;
        BLFacade blFacade = new BLFactory().getBusinessLogicFactory(isLocal);

        Seller s = blFacade.isLogged("seller1@gmail.com", "1234");
        if (s == null) {
            System.out.println("Login incorrecto: revisa email y password");
            return;
        }

        SellerTable st = new SellerTable(s);
        st.setVisible(true);
    }
}
```

Resultado:

